

CyberSens

Rapport complet de présentation de l'application

Sensibiliser • Protéger • Agir

Plateforme web de formation et de sensibilisation à la cybersécurité

État du projet au 29 septembre 2026

Sommaire

1. Résumé	3
2. Contexte, problème et objectifs	4
3. Fonctionnalités détaillées	4
4. Parcours utilisateur type	8
5. Architecture technique	8
6. Sécurité	10
7. Qualité, tests et livraison	11
8. Approche gestion de projet	12
9. Limites connues et pistes d'amélioration	14
10. Conclusion	15
Annexe A — Commandes utiles	15
Annexe B — Variables d'environnement	16

1. Résumé

CyberSens est une application web (installable sur téléphone et ordinateur) qui apprend la cybersécurité à un large public : particuliers, étudiants, professionnels et entreprises. Elle réunit dans un seul espace :

- **12 modules de formation** (60 leçons) avec quiz, examen final et **certificat vérifiable** ;
- **des quiz** générés par l'IA, et un **quiz multijoueur en salles** (code à 6 chiffres, chacun sur son téléphone) ;
- **une arène CTF** (Capture The Flag) de **24 défis** à drapeaux générés aléatoirement ;
- **9 mini-jeux** de défense, d'attaque simulée, de mémoire et de cryptographie ;
- **des Classements** (points, avec option de masquer son nom) et une **Communauté** (messages, commentaires, « j'aime », signalement) ;
- **6 outils de sécurité** (testeur de mot de passe, analyseur de liens, d'e-mails, de logs, détecteur de deepfake...) ;
- **un assistant IA** (Gemini) spécialisé en cybersécurité ;
- **un fil d'actualités** alimenté par de vrais flux RSS de sites spécialisés ;
- **trois langues** : français, anglais, espagnol.

Le projet est pensé pour être **sûr par conception** (serveur sans framework, clé IA jamais exposée au navigateur, examens corrigés côté serveur) et **déployable simplement** (Docker, Railway).

2. Contexte, problème et objectifs

2.1 Problème

La majorité des incidents de cybersécurité commencent par une erreur humaine : hameçonnage (phishing), mots de passe faibles, arnaques par SMS ou par appel, mauvaise configuration. Les formations existantes sont souvent longues, austères, en anglais et peu adaptées au contexte local (par exemple les arnaques Mobile Money en Afrique de l'Ouest, mises en avant dans l'application).

2.2 Objectifs du projet

Objectif	Traduction dans l'application
Rendre la cybersécurité accessible	Cours courts (40 à 83 min), niveaux Débutant / Intermédiaire / Avancé, 3 langues
Motiver à apprendre	Points, niveaux, badges, quiz, jeux, défis CTF
Prouver les acquis	Examen corrigé par le serveur, certificat signé et vérifiable publiquement
Aider au quotidien	Assistant IA, outils d'analyse, bonnes pratiques, actualités
Fonctionner partout	Application installable (PWA), utilisable hors ligne

2.3 Public visé (parties prenantes)

- **Apprenants** : particuliers, étudiants, professionnels, entreprises (le rôle est choisi à l'inscription).
- **Formateur / porteur du projet** : auteur des contenus (mentionné sur les certificats).
- **Vérificateurs** : toute personne qui contrôle l'authenticité d'un certificat (page publique).
- **Exploitant** : administrateur qui déploie, sauvegarde et supervise le service.

3. Fonctionnalités détaillées

L'application est organisée en onglets (**AppTab**), affichés dans une barre de navigation adaptée au bureau et au mobile.

3.1 Accueil

Point d'entrée avec un salut personnalisé, la progression, les recommandations de cours et des accès rapides vers les autres sections.

3.2 Apprendre (formations)

- **12 modules** répartis en trois niveaux :

Niveau	Modules
Débutant	1 Fondamentaux & réseau · 2 Authentification, cryptographie & identités
Intermédiaire	3 Ingénierie sociale & phishing · 4 Réseaux & navigation sécurisée · 5 Mobile & IoT · 10 CI/CD & conteneurs · 11 Infrastructure as Code & GitOps
Avancé	6 Entreprise, télétravail & PSSI · 7 IA : deepfakes, LLM, attaques adversariales · 8 Pentest éthique · 9 Forensics & réponse à incident (DFIR) · 12 SRE & observabilité

- **60 leçons** au total, chacune avec contenu, exemples, schéma, extrait de code, exercice pratique et question de compréhension.
- Filtres par domaine : fondations, réseaux, mobile, gouvernance, IA, offensif, DFIR, cloud.
- **Examen par module** (62 questions au total) : le score minimal pour réussir est de **70 %**. En cas de réussite, un **certificat** est émis.
- Laboratoires interactifs exécutés dans le navigateur ([CyberSensLabRunner](#)).

3.3 Quiz

- **Multijoueur** : un joueur crée une salle (5, 10 ou 15 questions, 15 à 30 s par question) et partage le code ou le lien ; les autres le rejoignent depuis leur téléphone (jusqu'à 12). Le serveur pilote le minuteur, la correction et le score (plus la réponse est rapide, plus elle rapporte, de 500 à 1 000 points) ; les bonnes réponses ne sont envoyées qu'à la correction, et chaque joueur lit les questions dans sa langue. 30 questions en trois langues.
- **Solo** : questions générées par l'IA (avec questions de secours si l'IA est indisponible), difficulté réglable, historique des scores.

3.4 Arène CTF

Un CTF (Capture The Flag) est un jeu où l'on cherche un « drapeau » caché en analysant des indices. L'arène propose **24 défis** répartis en cinq catégories, avec quatre niveaux de difficulté (Facile, Moyen, Difficile, Expert) et une valeur en points :

Catégorie	Exemples de défis
Web & Injection	Fuite d'en-têtes HTTP, injection SQL, falsification de JWT, phishing kit, inclusion de fichier local (LFI), XSS stocké, IDOR, injection de commande, fichier de sauvegarde exposé
Crypto & Obfuscation	Base64, ROT13, chiffrement XOR
IA & Prompt Injection	Jailbreak d'un agent IA, empoisonnement d'un système RAG
Forensic & Reverse	Logs SSH, désobfuscation hexadécimale, exfiltration DNS, commande PowerShell encodée
Système	Bucket S3 public, fatigue MFA, escalade de privilèges Linux, attaque de chaîne d'approvisionnement, secret Docker, secret Kubernetes

Chaque défi propose deux **indices progressifs**, et son drapeau change à chaque génération (impossible de copier la réponse d'un autre joueur). Un défi peut être régénéré individuellement.

3.5 Jeux (« Menaces IA »)

Neuf mini-jeux, présentés dans un hub organisé : **Chasse aux nouvelles menaces IA**, **DDoS Defense** (garder un serveur en ligne), **Lab Brute Force**, **Lab Injection SQL**, **Attaque Hacker** (réflexes), **Phishing** (repérer les messages piégés), **Pare-feu** (écrire les bonnes règles), **Mémoire** (associer menaces et parades) et **Chiffrement** (déchiffrer des messages).

3.6 Outils de sécurité

Six outils : testeur de deepfakes, audit de logs par IA, testeur de liens douteux, analyseur d'e-mails de phishing, testeur de mot de passe (entropie et résistance), journal d'audit local.

3.7 Assistant IA « CyberGuard »

Chat spécialisé (phishing, mots de passe, mobile, réseau, code, menaces IA, aide CTF). Il gère la saisie vocale, la lecture à voix haute, l'envoi de photos (analyse de documents ou de captures) et l'export de la conversation.

3.8 Actualités

Fil d'actualités cyber alimenté en **temps réel** par les flux RSS de six sites : CERT-FR, ZATAZ, LeMagIT, The Hacker News, BleepingComputer, Krebs on Security. Le serveur les agrège et les garde 10 minutes en cache. Les articles s'ouvrent avec un lien vers la source originale. Si aucune source ne répond, l'application affiche des articles locaux de secours et l'indique.

3.9 Bonnes pratiques, Profil, À propos, Don

- **Bonnes pratiques** : fiches de réflexes de sécurité.
- **Profil** : informations du compte, points, niveau, **badges** (8 badges dynamiques, avec dates et progression réelles), certificats, préférences (thème clair ou sombre, langue).
- **À propos** : présentation du projet et de son développeur, VDPHACKER (pentesteur junior, formateur en cybersécurité et IA).
- **Faire un don** : fiche de soutien (lien vers le livre) affichée aussi à l'accueil, plus un panneau latéral discret toutes les minutes (15 secondes, fermable, option « Ne plus afficher aujourd'hui »).
- **Compte** : changement de mot de passe dans le profil ; réinitialisation par l'exploitant (`npm run admin:reset-password -- <email>`).

3.10 Classements et Communauté

- **Classements** : palmarès par points. Seuls le prénom et l'initiale du nom sont affichés ; chaque membre peut se retirer du classement dans son profil. Le serveur plafonne les points déclarés à **1 500 par jour et par membre**.
- **Communauté** : messages par thème, commentaires, « j'aime ». Chaque message peut être **signalé** ; les administrateurs voient les signalements et peuvent supprimer tout contenu.

3.11 Centre DevOps (administrateurs)

Tableau de bord d'exploitation : état du service, mémoire, nombre d'inscrits, déclenchement d'une sauvegarde, purge des sessions expirées. **Réservé** aux comptes listés dans `ADMIN_EMAILS`.

4. Parcours utilisateur type

1. L'utilisateur ouvre l'application et **crée un compte** (mot de passe d'au moins 12 caractères).
 2. Il suit un **parcours d'accueil** (onboarding).
 3. Il choisit un module, lit les leçons, fait les exercices : chaque leçon terminée est **enregistrée sur le serveur**.
 4. Il passe l'**examen** ; si le score atteint 70 %, il reçoit **150 points** et un **certificat** numéroté.
 5. Il s'entraîne avec les quiz, les jeux et le CTF, et gagne des points, des niveaux et des badges.
 6. Un tiers peut **vérifier le certificat** via son numéro (contrôle de la signature).
-

5. Architecture technique

5.1 Vue d'ensemble

Navigateur (React, PWA)

├ /	fichiers statiques (dist/)
├ /api/...	comptes, progression, examens, certificats, actualités, admin
└ /api/gemini	relais IA authentifié



Serveur Node.js (sans framework)

├ SQLite (fichier data/cybersens.db)
├ Gemini (Google), avec repli automatique
└ Flux RSS externes (actualités)

5.2 Technologies

Couche	Choix	Justification
Interface	React 19, Vite 6, Tailwind CSS 3, lucide-react (icônes)	Écosystème mature, build rapide
Application installable	PWA (vite-plugin-pwa, Workbox)	Hors ligne, installation Android, iOS, Windows, macOS
Serveur	Node.js $\geq$ 22.18, module <code>node:http</code> , aucun framework	Surface d'attaque réduite, aucune dépendance npm à l'exécution
Base de données	SQLite natif (<code>node:sqlite</code>)	Simple, fichier unique, sauvegarde à chaud
IA	Google Gemini via relais serveur	La clé reste côté serveur
Qualité	TypeScript, ESLint, Prettier, Husky, lint-staged	Code cohérent et vérifié avant chaque commit
Livraison	Docker, Railway, GitHub Actions, Dependabot	Déploiement reproductible et automatisé

5.3 Organisation du code

Dossier	Contenu
<code>features/</code>	Un dossier ou fichier par écran (Home, Learn, Quiz, CTF, Games, Tools, AIChat, News, Profile, DevOps...)
<code>components/</code>	Éléments partagés (mise en page, modales, badges, indicateurs hors ligne, installation PWA)
<code>services/</code>	Logique métier côté navigateur : client d'API, authentification, persistance locale, IA, traductions, données des cours
<code>server/</code>	API, base de données, relais Gemini, sécurité (CSRF, en-têtes), actualités, sauvegarde
<code>tests/</code>	Tests d'intégration de l'API et des traductions
<code>docs/</code>	Checklists de mise en production et d'expérience utilisateur, présent rapport

5.4 Base de données

Huit tables : **users** (comptes), **sessions**, **lesson_progress** (leçons terminées), **certificates**, **quiz_results**, **login_attempts** (verrouillage), **security_log** (journal de sécurité) et **meta** (paramètres internes, dont le secret de signature). Les migrations sont successives et jamais modifiées après livraison. Toutes les requêtes SQL sont **préparées**.

5.5 Principe « le serveur fait foi »

Le stockage du navigateur n'est qu'un cache. Le serveur :

- corrige les **examens** (le navigateur ne peut pas se donner une bonne note) ;
- **signe** les certificats avec HMAC-SHA256 ;
- valide que les leçons existent avant de les enregistrer.

Les certificats importés depuis d'anciennes données locales sont marqués « importés, non vérifiés par examen serveur ».

5.6 Mode hors ligne et synchronisation

Le service worker met en cache l'application, les polices et les cours. Hors ligne, la progression, les points et les quiz sont mis en file d'attente et renvoyés au retour du réseau. L'examen de certification et l'assistant IA exigent une connexion.

5.7 Internationalisation

Trois langues (FR, EN, ES). Le contenu des cours est traduit par fichiers dédiés, et les textes de l'interface par un dictionnaire, avec des tests qui vérifient la couverture des traductions.

6. Sécurité

La sécurité est un critère central, cohérent avec le sujet de la plateforme.

Domaine	Mesure
Mots de passe	Hachage scrypt (paramètres OWASP), 12 caractères minimum, refus si le mot de passe contient le nom ou l'e-mail
Sessions	Jeton aléatoire, haché en base, cookie HttpOnly , SameSite=Lax , Secure derrière HTTPS, expiration à 30 jours

Domaine	Mesure
Attaques par force brute	Limitation de débit par IP, verrouillage 15 min après 5 échecs , message identique que l'e-mail existe ou non (comparaison à temps constant, empreinte factice)
CSRF	Vérification de l'origine, JSON obligatoire, refus des requêtes inter-sites sans en-tête Origin
Injection	Requêtes SQL préparées, validation stricte de toutes les entrées, chemins de fichiers normalisés (anti-traversée)
Navigateur	CSP stricte (aucun script ni style en ligne), X-Frame-Options: DENY , nosniff , HSTS, Permissions-Policy
IA	Clé Gemini uniquement côté serveur, relais réservé aux utilisateurs connectés, quota par compte
Administration	Routes DevOps réservées aux e-mails de ADMIN_EMAILS (désactivées si la variable est vide)
Certificats	Signature HMAC vérifiée en temps constant ; le secret CERT_SECRET doit être différent entre test et production
Traçabilité	Table security_log : inscriptions, connexions, échecs
Dépendances	npm audit : aucune vulnérabilité connue en production ; Trivy et Dependabot en intégration continue

Un audit de sécurité a été mené pendant ce projet ; il a révélé et corrigé une faille critique (routes d'administration ouvertes sans authentification).

7. Qualité, tests et livraison

7.1 Tests

- **34 tests automatisés, tous verts** : API (santé, inscription, progression, examen et certificat signé, sécurité CSRF et injections, connexion et verrouillage, administration, quota IA, classement, communauté, plafond de points), comptes (changement et réinitialisation de mot de passe), panneau de don, traduction des cours (français, anglais, **espagnol : 12 modules alignés**) et de l'interface.

- Vérifications automatiques : TypeScript (**tsc**), ESLint (aucune erreur), Prettier, hooks Git.

7.2 Intégration et déploiement continu

`.github/workflows/ci.yml` exécute à chaque push : typage, tests, build, audit npm, recherche de clé d'API dans le build, construction et test de l'image Docker, analyse Trivy.

7.3 Environnements

Deux environnements isolés sur Railway : **staging** (branche `staging`, vérification) et **production** (branche `main`), chacun avec sa base et son volume.

7.4 Conteneur

Image Docker en deux étapes ; exécution sans privilèges (utilisateur non root, système de fichiers en lecture seule, capacités retirées, mémoire limitée), **contrôle de santé** sur `/api/health`.

7.5 Exploitation

Journal d'accès JSON (méthode, chemin, statut, durée : jamais de corps ni de cookie), sauvegarde à chaud (`npm run db:backup`), restauration documentée, arrêt propre sur **SIGTERM**.

8. Approche gestion de projet

8.1 Périmètre livré (MVP étendu)

Formation complète avec certification, gamification (points, niveaux, badges, quiz, jeux, CTF), outils, assistant IA, actualités en direct, multilingue, PWA, déploiement automatisé.

8.2 Cycle et méthode

Développement itératif par petites livraisons (commits ciblés : correctifs, ajouts de contenu, améliorations d'interface), avec vérification automatique à chaque étape (typage, lint, tests). Le dépôt Git suit une stratégie `feature` → `staging` → `main`.

8.3 Risques identifiés et réponses

Risque	Gravité	Réponse en place ou recommandée
Indisponibilité ou surcharge de l'IA (erreurs 503)	Élevée	Nouvel essai automatique puis modèles de secours ; repli sans streaming ; questions de secours pour les quiz
Faible de sécurité (accès non autorisé)	Critique	Audit, correctifs, tests dédiés, CSP, limitation de débit, journal de sécurité
Perte de données	Élevée	Volume persistant, sauvegarde à chaud, procédure de restauration (à automatiser par cron)
Dépendance à des sites externes (actualités)	Moyenne	Cache, timeout, sources multiples, repli sur des articles locaux
Fraude sur les certificats	Moyenne	Signature HMAC, vérification publique, examen corrigé côté serveur
Contenu obsolète	Moyenne	Actualités en direct ; mise à jour régulière des modules à planifier
Coût de l'IA	Moyenne	Quotas par compte, limitation de débit

8.4 Postes de coûts à suivre

Hébergement (Railway ou VPS avec disque persistant), nom de domaine et certificat HTTPS, appels à l'API Gemini (variables selon l'usage), temps de rédaction et de mise à jour des contenus, traduction et relecture, maintenance de sécurité. Aucun montant n'est donné ici : il dépend de l'hébergeur choisi et du nombre d'utilisateurs.

8.5 Indicateurs de réussite proposés

Taux de complétion des modules, taux de réussite aux examens (seuil 70 %), nombre de certificats émis, utilisateurs actifs par semaine, temps moyen jusqu'au premier module terminé, disponibilité du service, nombre d'incidents de sécurité.

9. Limites connues et pistes d'amélioration

9.1 Limites actuelles (transparence)

1. Le quiz multijoueur garde ses salles en mémoire (une seule instance ; un redémarrage interrompt les parties en cours) et ne crédite pas de points au classement.
2. Connexion Google et « mot de passe oublié » par e-mail : fonctionnels seulement une fois configurés (`GOOGLE_CLIENT_ID`, Resend) ; sans configuration, la réinitialisation passe par l'exploitant. Pas de vérification d'adresse e-mail à l'inscription.
3. Une seule instance possible (SQLite et limites de débit en mémoire).
4. L'image Docker n'a pas été construite en local ; la CI s'en charge.
5. L'inscription révèle si un e-mail existe déjà (compromis assumé, limité à 10 essais par heure et par IP).
6. Les vignettes des actualités utilisent une image générique (les images des sites externes ne sont pas chargées pour respecter la CSP).
7. Un scan de sécurité dynamique (HawkScan) n'a pas été exécuté faute de clé.

9.2 Pistes d'évolution

- Vrai multijoueur en temps réel (WebSocket).
 - Vérification d'adresse e-mail à l'inscription.
 - Tableau de bord pour les entreprises (suivi d'équipes, rapports).
 - Refonte visuelle avec un système de design commun (couleurs, composants, accessibilité).
 - Sauvegardes planifiées automatiquement et alertes de supervision.
 - Nouveaux modules et défis CTF (stéganographie, analyse réseau, sécurité mobile).
-

10. Conclusion

CyberSens couvre l'ensemble d'un parcours d'apprentissage en cybersécurité, de la découverte à la certification, avec une forte composante pratique (CTF, laboratoires, outils, IA). Son architecture volontairement simple (Node sans framework, SQLite, PWA) la rend légère, sûre et facile à déployer. Les principaux chantiers restants concernent le vrai multijoueur, la fiabilisation des tests de traduction, l'automatisation des sauvegardes et une refonte visuelle homogène.

Annexe A — Commandes utiles

Commande	Rôle
<code>npm install</code>	Installer les dépendances
<code>npm run dev</code>	Lancer l'application en développement (http://localhost:3000)
<code>npm run check</code>	Lint, typage, tests et build avant livraison
<code>npm run build</code> puis <code>npm start</code>	Production
<code>npm run db:backup</code>	Sauvegarde de la base
<code>docker compose up -d --build</code>	Déploiement Docker

Annexe B — Variables d'environnement

Variable	Rôle
GEMINI_API_KEY	Clé de l'assistant IA (jamais envoyée au navigateur)
CERT_SECRET	Secret de signature des certificats (unique par environnement)
ADMIN_EMAILS	E-mails des administrateurs (Centre DevOps)
GEMINI_FALLBACK_MODELS	Modèles IA de secours (facultatif)
PORT, HOST	Adresse d'écoute du serveur
TRUST_PROXY	1 derrière un reverse-proxy HTTPS (HSTS, cookies sécurisés)
DB_PATH, BACKUP_DIR	Emplacement de la base et des sauvegardes
ACCESS_LOG	0 pour désactiver le journal d'accès